

System-Level Analysis of Optical Circuit Switching Networks to Train Large Language Models

Arno Troch*, Abubakr Nada†, Michael Peeters†, Jeroen Famaey*

*IDLab, University of Antwerp - imec, Antwerp, Belgium. Email: firstname.lastname@uantwerpen.be

†imec, Leuven, Belgium. Email: firstname.lastname@imec.be

Abstract—Training Large Language Models (LLMs) increasingly stresses Graphics Processing Unit (GPU) interconnect networks. Today, these networks are based on Electrical Packet Switching (EPS), which can become a scaling bottleneck due to constraints in bandwidth density and power efficiency. Emerging optical I/O and Optical Circuit Switching (OCS) technologies promise to reshape the network design space by enabling an order of magnitude higher per-GPU bandwidth and switch port radix, potentially allowing interconnection of all GPUs using a single unified network. In this paper, we study how such a unified, high-capacity OCS network affects LLM training. We develop a full system-level design, from device architecture to circuit reconfiguration strategy, and evaluate a range of LLM training workloads using an analytical performance model. Our results show a twofold reduction on average in iteration times compared to state-of-the-art EPS systems, and also reveal how the network architecture changes the optimal parallelism strategy, providing important insights for future LLM training systems.

Index Terms—optical IO, optical circuit switching, large language model, distributed training, hardware-software co-design

I. INTRODUCTION

Over the past five years, large-scale generative Artificial Intelligence (AI) has grown exponentially, with frontier Large Language Models (LLMs) doubling in parameter count every ten months and now reaching multi-trillion-parameter scales [1]. Training these models requires an enormous amount of computational power and High Bandwidth Memory (HBM) capacity, far exceeding the capabilities of a single Graphics Processing Unit (GPU). Therefore, large-scale LLM training is performed on clusters with tens to hundreds of thousands of GPUs, using various parallelism strategies that introduce frequent inter-GPU communication to exchange intermediate activations, gradients, and parameters. As a result, LLM training performance is tightly coupled to inter-GPU communication efficiency and the underlying network architecture.

Today, state-of-the-art GPU clusters are typically composed of two Electrical Packet Switching (EPS)-based networks [2]: a *scale-up network* (e.g., NVLink) that offers high bandwidth within a rack, and a *scale-out network* (e.g., Ethernet or InfiniBand) that interconnects racks through multi-level Clos-based topologies. However, as the clusters grow, EPS faces several scaling issues [3]. First, electrical switches consume significant power and are reaching fundamental I/O and bandwidth limits. Second, packet switching introduces problems such as queueing delays and congestion, which are amplified by the bursty and non-uniform traffic patterns generated in LLM training.

As a result, optical I/O and Optical Circuit Switching (OCS) promise to reshape future GPU interconnects [3]. Unlike today's electrical networks, optical I/O enables significantly higher bandwidth density and larger switch port radix due to smaller per-port I/O footprint. In principle, such bandwidth and radix gains make it possible to interconnect all GPUs in a cluster using a single, unified OCS-based network, eliminating the distinction between scale-up and scale-out networks.

Recent studies on OCS for distributed training have primarily focused on 1) physical network topologies and architectures; and 2) circuit reconfiguration strategies. SiP-ML [4] and Light-wave Fabrics [5] demonstrate that OCS networks combined with intelligent workload mapping can significantly improve distributed training performance. In parallel, works such as TopoOpt [6], TOPO-X [7], ACTINA [8], and PCCL [9] show how the ability to reconfigure circuits can be exploited to optimize the network topology for communication patterns arising from distributed training.

In contrast to these efforts, in this paper, we study the *system-level implications* of a unified, high-capacity OCS fabric with optical I/O. Specifically, we aim to explore how this shift in network architecture affects not only LLM training performance, but also the optimal parallelism strategy used to train the model. Our contributions are as follows.

- We present a full system architecture with a unified OCS network, eliminating the traditional scale-up/scale-out split in EPS-based networks.
- We introduce a lightweight reconfiguration strategy derived from the one-shot approach presented in [8]. We replace the resource allocation heuristic with a heuristic based on the *network arithmetic intensity* (i.e., FLOPs per communicated byte) of each parallelism type.
- Through analytical evaluation of different LLM training workloads, we demonstrate that a unified OCS fabric not only reduces training iteration time ($1.42\times$ – $2.43\times$ over state-of-the-art EPS systems), but also *changes the optimal parallelism strategy*.

Our results highlight the importance of co-designing future network architectures and training workloads. The remainder of this paper is structured as follows. Section II presents our system model and assumptions. Section III describes our circuit reconfiguration and evaluation methodology. Sections IV and V present the results and discuss their implications. Finally, Section VI concludes and outlines future work.

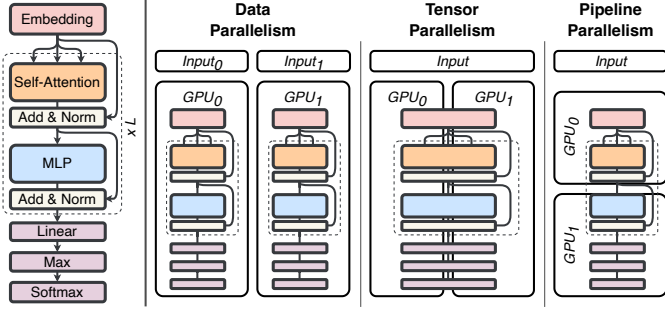


Fig. 1: High-level depiction of a decoder-only Transformer architecture (left) and the main parallelism strategies used to split a large Transformer across multiple GPUs (right).

II. SYSTEM MODEL

In this section, we describe the system model that we use for our evaluations. We start by providing a brief introduction to the LLM architecture and the corresponding types of parallelism used to train an LLM on many GPUs. Then, we describe the baseline and our proposed system architecture and state our assumptions. Finally, we introduce the analytical model used to evaluate the performance of LLM training on the two systems.

A. Transformers, Parallelism, and Collectives

Transformers are the core architecture underlying most modern LLMs. As illustrated in Fig. 1, a Transformer model is composed of a stack of identical layers, each consisting of two main components: an attention block and a Multi-Layer Perceptron (MLP) block. Training such models at large scale requires parallelism strategies to fit the model in GPU memory and speed up training time by distributing computation across many GPUs. The three main types of parallelism used are shown in Fig. 1. Data Parallelism (DP) [10] replicates the entire model across GPUs and distributes different subsets of training data among them. A widely used variant called Fully Sharded DP (FSDP) also reduces memory usage by *sharding* (i.e., partitioning) model parameters, gradients, and/or optimizer states across devices rather than fully replicating them. Tensor Parallelism (TP) [11] partitions each individual model layer across GPUs, which introduces significant communication overhead. Pipeline Parallelism (PP) [12] divides the model layers into sequential stages across GPUs, requiring less communication than TP, but at the cost of *pipeline bubbles*, or idle periods where some stages are inactive while others are computing. In practice, these three forms of parallelism are often used together [11], [13].

To illustrate the scale of modern LLM training, consider the Llama-3 405B model [14], which was trained on 16 384 NVIDIA H100 GPUs. In one of its pre-training steps, the model was trained using TP=8, PP=16, and DP=128, with a batch size of 2048 samples and a sequence length of 8192, corresponding to 16 million tokens per batch. The entire model was trained on 15.6 trillion tokens, requiring 3.8×10^{25} FLOPs. At this scale, the parallelism scheme and the inter-GPU communication significantly affects the training performance.

Each of these types of parallelism uses a distinct set of *collective operations*, such as *All-Gather*, *Reduce-Scatter*, and *All-Reduce*, to exchange data shards across GPUs. In TP, the model’s tensors are split across GPUs, requiring frequent exchange of intermediate activations via an *All-Reduce* (or an *All-Gather + Reduce-Scatter*) at every single layer, in order to assemble distributed tensors or combine and repartition partial results. In DP, model replicas process different input batches independently during the forward pass and perform *All-Reduce* collectives in the backward pass to aggregate partial gradients (i.e., sum and average) between all participating GPUs. FSDP also shards the model parameters and optimizer states across GPUs, introducing additional *All-Gathers* in the forward pass to combine parameters locally and *Reduce-Scatters* in the backward pass to distribute gradient updates. Finally, in PP, the model layers are placed on different GPUs, and communication consists of relatively small point-to-point transfers of intermediate activations between adjacent stages.

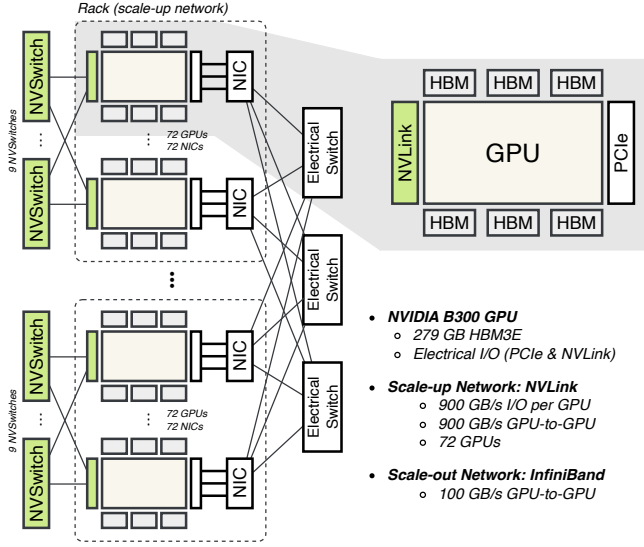
An important practical observation is that, while each of these collectives serves a different purpose, they are often implemented by forwarding data in a ring [15] because it is simple and works well across heterogeneous systems. In such a ring, each GPU communicates with only two neighbors, sending data shards to its successor and receiving shards from its predecessor. We exploit this in our OCS-based system, as discussed in Section III.

B. System Architecture

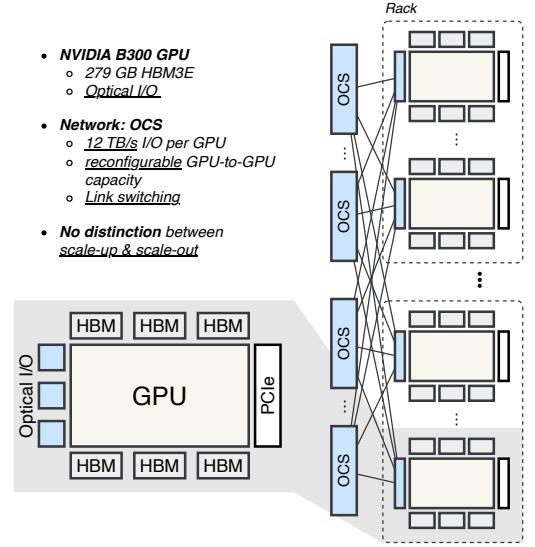
Fig. 2 shows the architecture of the systems that we aim to compare. The baseline system, depicted in Fig. 2a, is modeled after NVIDIA’s state-of-the-art commercially available GB300/NVL72 clusters [16]. This system contains two different EPS-based networks: a high-bandwidth *scale-up network*, connecting a small amount of GPUs within a rack, and a lower-bandwidth *scale-out network*, interconnecting racks of GPUs. The scale-up network is modeled after NVIDIA NVLink, connecting 72¹ B300 GPUs using electrical copper cables to 9 EPS-based NVSwitches, providing 7.2Tbps bi-directional bandwidth per GPU. The scale-out network represents NVIDIA’s InfiniBand network [17]. This network connects each GPU’s Network Interface Card (NIC) to a set of EPS-based InfiniBand switches using optical cables, with electrical-to-optical and optical-to-electrical transceivers enabling signal conversion between GPUs and the packet switches, providing 800Gbps GPU-to-GPU capacity. Note that the GPU capacity of the scale-out network is a factor of nine lower compared to the scale-up due to the range limits of electrical I/O [3].

In contrast, we propose the architecture in Fig. 2b. This architecture uses the same B300 GPUs, but eliminates the distinction between scale-up and scale-out networks by introducing a unified high-capacity OCS-based system, based on two key technologies: optical I/O and circuit switching. First, optical I/O can offer better bandwidth and energy efficiency compared to

¹For clarity, we evaluate a scale-up network with 64 GPUs rather than 72, since power-of-two system sizes are more familiar and improve readability.



(a) Baseline system, representing NVIDIA's GB300/NVL72 system [16]



(b) OCS system

Fig. 2: System architecture of the baseline and OCS system. For the OCS system, we propose to remove the distinction between scale-up and scale-out network. Instead, we use a single high-capacity OCS network with optical I/O at the GPU.

electrical links. Recent demonstrations of co-packaged silicon photonics indicate that optical I/O integrated with the chip can achieve bandwidth densities of several tens of Tbps/mm with much lower energy per bit [18], [19]. Second, circuit switching, unlike packet switching, establishes deterministic and congestion-free paths between GPUs, eliminating buffering and queueing delays at the switch. Additionally, optical switches can provide a higher radix and a higher bandwidth density, unconstrained by electrical I/O limitations.

To estimate the achievable GPU-to-GPU network capacity in the OCS system, we bound the total bandwidth by the amount of optical I/O that can be integrated along the edge of a GPU die. Because the die size of the B300 GPU has not been publicly disclosed, we approximate it using the previous-generation H100 GPU, which has a die area of 814 mm² [20]. Approximating the dimensions of the GPU by a 4 × 3 rectangular die, this corresponds to dimensions of 32.95 mm × 24.71 mm. For our analysis, we adopt the optical I/O chiplet design from [18], in which a single chiplet occupies 8.1 mm of die edge and provides 32 Tbps bidirectional bandwidth through 16 links, each carrying 64 wavelengths at 32 Gbps per wavelength. Under these assumptions, the short edge of the GPU die can accommodate three optical I/O chiplets in place of the existing NVLink interface, resulting in an aggregate per-GPU network capacity of 96 Tbps across 48 optical links. We assume that each of these links can be individually assigned to a specific circuit.

C. Analytical Model

To compare these systems, we extend *Calculon*² [21], an open-source analytical performance model for large-scale LLM

training. Given an LLM, hardware configuration, and parallelism strategy, *Calculon* calculates the training iteration time by analytically scheduling computation, memory, and communication costs. This allows us to capture how LLM architecture, parallelism strategy, and hardware interact.

As a building block, we model each collective operation using a ring pattern, as this is the most common implementation of collectives in practice [15]. The completion time of such a ring with n participating GPUs, total message size b bits, and link capacity C bits/s is

$$t_{\text{ring}} = \frac{(n-1) \cdot b}{n \cdot C}.$$

In each of the $n-1$ steps of the ring, every GPU sends b/n bits. The completion time of a ring *All-Gather* and *Reduce-Scatter* is equal to t_{ring} , while a ring *All-Reduce* requires $2 \cdot t_{\text{ring}}$ time: one ring to gather and sum the data shards, and one ring to distribute the results. Note that the value of b in these collectives is highly sensitive to both the type of parallelism in which it is used, as well as the architecture of the LLM.

In the baseline system, C is set to the effective GPU-to-GPU capacity of either the scale-up or the scale-out network, depending on which network the collective is performed on. For the OCS system, we instead express the capacity in terms of optical links l . The ring completion time for parallelism type p subject to the total amount of links $\mathcal{L}$ becomes

$$\begin{cases} t_{\text{ring},p} = \frac{(n_p-1) \cdot b_p}{n_p \cdot l_p \cdot C_l} & \forall p \in \{\text{tp}, \text{dp}, \text{pp}\} \\ \sum_{p \in \{\text{tp}, \text{dp}, \text{pp}\}} l_p = \mathcal{L} \end{cases}.$$

Each link provides capacity C_l , and we allocate l_p links to the circuit used by parallelism type $p \in \{\text{tp}, \text{dp}, \text{pp}\}$. n_p and b_p

²<https://github.com/calculon-ai/calculon>

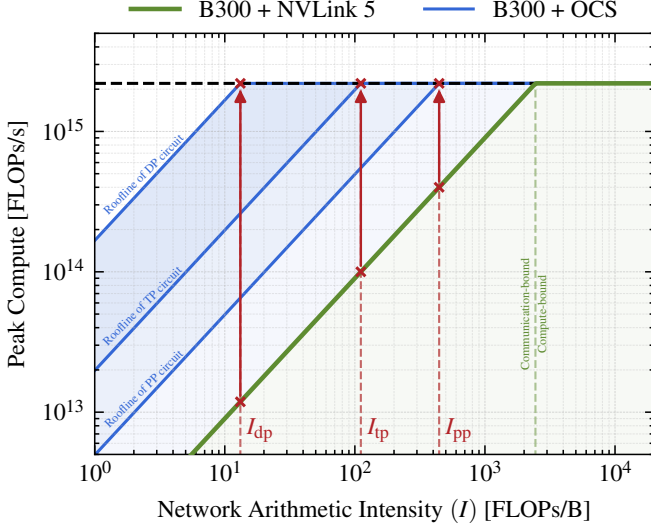


Fig. 3: Network rooflines for baseline and OCS systems. I_{dp} , I_{tp} , and I_{pp} are obtained from *Calculon* by training GPT-3 175B on 512 GPUs with $tp = 2, dp = 16, pp = 16$. With the OCS system, higher network capacity and parallelism-specific circuits allow us to approximately align each roofline ridge point with the corresponding arithmetic intensity.

represent the number of participants and the communication volume in bits of parallelism type p , respectively. We use this formulation in *Calculon* to evaluate how different link allocations across TP, DP, and PP affect end-to-end iteration time. The allocation of l_p is determined by the weighted allocation mechanism described in Section III-A.

III. METHODOLOGY

In this section, we introduce a one-shot optical circuit reconfiguration scheme based on two fundamental properties of LLM training workloads: 1) collective operations are often implemented as logical rings; and 2) different parallelism strategies have different *network arithmetic intensities*, defined as the number of FLOPs performed per byte communicated over the network (FLOPs/B). Since the network is reconfigured only once before training and remains fixed thereafter, reconfiguration latency does not impact workload completion time and is therefore not considered in this work. Additionally, we leave the physical topology design for future work. We first describe how we use these properties to create circuits and allocate optical links, and then outline our simulation setup.

A. Weighted Resource Allocation

We first determine how many distinct peers each GPU communicates with during LLM training. As discussed in Section II-A, both TP and DP rely on ring-based collectives, while PP uses point-to-point communication between adjacent pipeline stages. Assuming all three forms of parallelism are used, each GPU directly exchanges data with two neighbors in the TP ring, two in the DP ring, and at most two neighbors in PP (i.e., the previous and/or next pipeline stage). A GPU

Algorithm 1 Arithmetic intensity-based link allocation

Require: Total links $\mathcal{L}$,
Network arithmetic intensities I_{tp}, I_{pp}, I_{dp}
Ensure: Link allocation l_{tp}, l_{pp}, l_{dp} summing to $\mathcal{L}$

```

for  $p \in \{tp, pp, dp\}$  do
   $w_p \leftarrow 1/\sqrt{I_p}$ 
end for
 $S \leftarrow \sum_p w_p$ 
for  $p \in \{tp, pp, dp\}$  do
   $r_p \leftarrow \min(2 \cdot \lceil w_p \rceil, \mathcal{L} \cdot w_p / S)$   $\triangleright \geq 2$  if  $w_p \neq 0$ .
   $l_p \leftarrow \lfloor r_p \rfloor$ 
   $f_p \leftarrow r_p - l_p$ 
end for
 $R \leftarrow \mathcal{L} - \sum_p l_p$ 
if  $R > 0$  then  $\triangleright$  Distribute remaining links
  sort  $p \in \{tp, pp, dp\}$  by descending  $f_p$ 
  for  $k = 1$  to  $R$  do
     $l_{p_k} \leftarrow l_{p_k} + 1$ 
  end for
end if
return  $(l_{tp}, l_{pp}, l_{dp})$ 

```

therefore needs at most six logical connections, independent of system scale, meaning that six dedicated optical paths per GPU can satisfy all required connections.

To decide how much network capacity to assign to each parallelism type, we adopt the concept of *network roofline models* [22]. The roofline relates peak compute performance (FLOPs/s) to *network arithmetic intensity* (FLOPs/B). Fig. 3 shows the roofline of the EPS-based baseline system (green). The curve consists of a communication-bound region, where performance scales with network bandwidth, and a compute-bound region at the peak GPU FLOPs/s. Their intersection is the *ridge point*. Workloads with arithmetic intensity to the left of the ridge point are *communication-bound*, while workloads to the right are *compute-bound*. The example in Fig. 3 shows that, in the baseline system, all three arithmetic intensities I_{tp} , I_{pp} , and I_{dp} lie in the communication-bound region. In the OCS-based system (blue), optical I/O increases total network capacity, and circuit switching allows us to assign separate circuits to each parallelism domain, each with its own effective bandwidth and thus its own roofline. Subject to the finite total network capacity, our goal is to allocate bandwidth so that the ridge point of each parallelism domain p lies as close as possible to its arithmetic intensity I_p , reducing communication bottlenecks and improving GPU utilization.

To achieve this, we introduce a weighted link allocation scheme shown in Algorithm 1, inspired by the pre-planned reconfiguration strategy in [8]. The algorithm takes as input the total number of links $\mathcal{L}$ and the network arithmetic intensities I_p for each parallelism type p , computed with *Calculon* as FLOPs per iteration divided by the bytes sent over the network for that p . We then calculate the weights $w_p = 1/\sqrt{I_p}$ and distribute links proportional to w_p , assigning more capacity to more communication-intensive circuits while enforcing a

minimum of two links per parallelism type to form a ring. The choice of w_p follows from [8], stating that the allocation that minimizes the total transfer time is proportional to the square root of the communication volume of each circuit.

B. Simulation Setup

We evaluate the OCS system and the proposed link allocation method across different LLM training workloads, varying both model scale and system size, and comparing *training iteration time*. The *training iteration time* is the time to complete one full training iteration (forward pass, backward pass, and weight update), including computation and communication. We consider GPT-3 175B [23], Llama3 70B and 405B [14], and Megatron-LM models [13] with 40B to 1T parameters. We evaluate cluster sizes from 16k to 100k GPUs, reflecting the scale of frontier LLMs training systems. For each combination of model and hardware system (baseline or OCS), we use *Calculon* to determine all valid parallelism schemes (i.e., each valid combination of TP, DP, and PP) and compute the corresponding iteration time. In the OCS case, we first compute the network arithmetic intensities I_p for each parallelism type $p \in \{tp, pp, dp\}$, apply Algorithm 1 to obtain link allocations l_p , and use these allocations in our analytical communication model. For each training job, after exhaustively searching all valid schemes, we report the lowest iteration time on each system and the parallelism scheme that achieves it. This quantifies both the raw performance impact of using OCS, as well as how the network architecture affects the optimal choice of parallelism. To validate our allocation scheme, we also compare our arithmetic intensity-based allocation with an *equal-share* OCS system that divides links evenly across parallelism types.

IV. RESULTS

Table I summarizes the best iteration times obtained for a range of LLMs and system sizes. For each configuration, we report the optimal combination of TP, DP (or FSDP), and PP on three systems: the EPS-based baseline (*base*), the OCS system with equal link distribution across parallelism types (*ocs-eq*), and the OCS system using arithmetic intensity-based link allocation (*ocs-ai*). The OCS systems outperform the baseline across all evaluated models and system scales, with speedups of $1.42\times$ – $2.43\times$. It is clear that most of the speedup comes from the significant increase in network capacity enabled by optical I/O, but the arithmetic intensity-based allocation method still provides a modest but consistent improvement over the equal-share OCS system, showing that even a simple workload-aware allocation can provide measurable benefits.

The optimal choice of parallelism also changes noticeably between systems. In the baseline system, the best configurations often favor a higher degree of PP. In contrast, the OCS systems reduce the degree of PP, increase the degree of DP, and move from regular DP to FSDP. FSDP reduces memory use by sharding parameters, gradients, and optimizer states across devices, but introduces additional communication during forward and backward passes. In the baseline system, which has a relatively low-capacity scale-out network, this additional communication

TABLE I: Best parallelism scheme and corresponding training iteration time for various LLMs on baseline and OCS systems.

Model	#GPU	System	TP	DP	PP	Iteration time (ms)	Speedup
Megatron-40B	16384	<i>base</i>	8	256	8	133.2	
		<i>ocs-eq</i>	16	1024*	1	57.1	$2.33\times$
		<i>ocs-ai</i>	16	1024*	1	54.7	$2.43\times$
Llama3-70B ¹	16384	<i>base</i>	8	256	8	208.1	
		<i>ocs-eq</i>	8	512*	4	137.5	$1.51\times$
		<i>ocs-ai</i>	8	512*	4	135.0	$1.54\times$
Megatron-145B	49152	<i>base</i>	16	384	8	264.6	
		<i>ocs-eq</i>	16	768*	4	155.5	$1.70\times$
		<i>ocs-ai</i>	16	768*	4	150.9	$1.75\times$
GPT-3-175B	49152	<i>base</i>	16	256	12	311.9	
		<i>ocs-eq</i>	16	768*	4	185.9	$1.68\times$
		<i>ocs-ai</i>	16	768*	4	180.4	$1.73\times$
Megatron-310B	61440	<i>base</i>	32	320	6	347.4	
		<i>ocs-eq</i>	32	640*	3	183.7	$1.89\times$
		<i>ocs-ai</i>	32	640*	3	175.6	$1.98\times$
Llama3-405B ¹	71680	<i>base</i>	8	640	14	785.8	
		<i>ocs-eq</i>	8	640*	14	553.9	$1.42\times$
		<i>ocs-ai</i>	8	640*	14	546.4	$1.44\times$
Megatron-530B	81920	<i>base</i>	32	512	5	511.3	
		<i>ocs-eq</i>	32	512*	5	269.0	$1.90\times$
		<i>ocs-ai</i>	32	512*	5	257.4	$1.99\times$
Megatron-1T	98304	<i>base</i>	32	192	16	803.1	
		<i>ocs-eq</i>	32	768*	4	453.6	$1.77\times$
		<i>ocs-ai</i>	32	768*	4	436.9	$1.84\times$

Best parallelism strategy and corresponding iteration time for LLMs on EPS (*base*), equal-share OCS (*ocs-eq*), and arithmetic intensity-based OCS (*ocs-ai*) systems. TP/DP/PP are tensor/data/pipeline degrees, with $TP \cdot DP \cdot PP = \#GPU$. “Speedup” is the ratio between iteration times of *base* and *ocs-xx*. * FSDP is used instead of regular DP, i.e., parameters, gradients, and optimizer states are sharded rather than fully replicated.

¹ Llama3 models use Grouped Query Attention (GQA) [14], [24] for the attention block with 8 key-value heads, limiting the TP degree to 8.

makes FSDP an unattractive means of reducing the memory footprint of a model. Instead, the model is split across GPUs using PP, which introduces significantly less communication, but at the cost of pipeline bubbles. In the OCS systems, the communication overhead introduced by FSDP can be tolerated thanks to the unified OCS network, which has much higher network capacity and dedicated circuits. The resulting memory savings, combined with the larger DP degree and fewer pipeline stages, in turn reduce the iteration time.

V. DISCUSSION

Our findings highlight that future large-scale LLM training systems will need to reconsider how parallelism strategies are chosen, as these choices become tightly coupled to the underlying network architecture. By moving from a heterogeneous EPS network to a unified high-capacity OCS network, the balance between computation and communication is altered. As a result, conventional best practices (e.g., [13]) that guide the choice of TP, PP, and DP degrees may no longer hold. Parallelism schemes that are too communication-heavy in today’s systems, such as strategies that use higher degrees of DP or FSDP, become much more attractive. Similarly, parallelism schemes

with minimal communication overhead, such as PP, may no longer provide optimal results.

Some of our assumptions limit the practical applicability of the proposed design. First, a practical consideration is that sustaining our assumed 96 Tbps per-GPU optical I/O requires an equally high die-to-die (D2D) bandwidth to feed the optical chiplets. Although this bandwidth density is challenging for current packaging technologies and more advanced integration techniques would be required, this is a practical rather than a fundamental limitation of the proposed architecture. Second, while our link allocation strategy is based on network arithmetic intensity, it does not explicitly account for whether communication lies on the critical path or can be overlapped with computation. For example, DP communication can often be hidden behind backpropagation, whereas TP collectives are typically blocking. Accounting for these overlaps in resource allocation is an important direction for future work.

VI. CONCLUSION & FUTURE WORK

In this work, we examine how a unified, high-capacity OCS network with optical I/O reshapes the design space of large-scale LLM training systems. By creating a full system-level model, from GPU-level optical I/O assumptions to link allocation, and evaluating different training workloads on varying system sizes, we demonstrate that OCS fabrics provide significant benefits in iteration time compared to state-of-the-art EPS networks. Beyond raw performance, our results reveal that changing the network architecture significantly affects the effectiveness of different parallelism schemes, with lower degrees of PP and higher degrees of DP becoming more favorable.

For future work, we plan to extend the allocation algorithm to explicitly account for compute–communication overlap. Moreover, integrating the proposed OCS model into a detailed system-level simulator would allow empirical validation of our analytical findings and further inform the co-design of future OCS-enabled LLM training systems.

REFERENCES

- [1] Epoch AI, “Data on AI Models,” Jul. 2025. [Online]. Available: <https://epoch.ai/data/ai-models>
- [2] Marvell, “The evolution of AI interconnects,” Mar. 2025. [Online]. Available: <https://www.marvell.com/blogs/the-evolution-of-ai-interconnects.html>
- [3] Ayar Labs, “In-Package Optical I/O for Generative AI Architectures,” Jan. 2024. [Online]. Available: <https://ayarlabs.com/resources/in-package-optical-i-o-for-generative-ai-architectures/>
- [4] M. Khani, M. Ghobadi, M. Alizadeh, Z. Zhu, M. Glick, K. Bergman, A. Vahdat, B. Klenk, and E. Ebrahimi, “SiP-ML: High-bandwidth optical network interconnects for machine learning training,” in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, ser. SIGCOMM ’21. New York, NY, USA: Association for Computing Machinery, Aug. 2021, pp. 657–675, doi: 10.1145/3452296.3472900.
- [5] H. Liu *et al.*, “Lightwave Fabrics: At-Scale Optical Circuit Switching for Datacenter and Machine Learning Systems,” in *Proceedings of the ACM SIGCOMM 2023 Conference*. New York NY USA: ACM, Sep. 2023, pp. 499–515, doi: 10.1145/3603269.3604836.
- [6] W. Wang, M. Khazraee, Z. Zhong, M. Ghobadi, Z. Jia, D. Mudigere, Y. Zhang, and A. Kewitsch, “TopoOpt: Co-optimizing Network Topology and Parallelization Strategy for Distributed Training Jobs,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 739–767. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/wang-weiyang>
- [7] Y.-X. Hu, H. Tian, Y. Zhao, F. Wu, and X.-Y. Li, “TOPO-X: Co-optimize Flow Scheduling, Topology, and ML Training Parallelism,” in *2025 34th International Conference on Computer Communications and Networks (ICCCN)*, Aug. 2025, pp. 1–6, doi: 10.1109/ICCCN65249.2025.11133974.
- [8] Z. Wu, B. Klenk, L. Dennison, and K. Bergman, “ACTINA: Adapting Circuit-Switching Techniques for AI Networking Architectures,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. St. Louis MO USA: ACM, Nov. 2025, pp. 1211–1222, doi: 10.1145/3712285.3759842.
- [9] A. V. Kumar, A. Devraj, and R. Singh, “PCCL: Photonic circuit-switched collective communication for distributed ML,” Sep. 2025, doi: 10.48550/arXiv.2509.15450.
- [10] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory optimizations toward training trillion parameter models,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’20. Atlanta, Georgia: IEEE Press, Nov. 2020, pp. 1–16. [Online]. Available: <https://dl.acm.org/doi/10.5555/3433701.3433727>
- [11] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” Mar. 2020, doi: 10.48550/arXiv.1909.08053.
- [12] Y. Huang *et al.*, “GPipe: Efficient training of giant neural networks using pipeline parallelism,” in *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., Dec. 2019, no. 10, pp. 103–112. [Online]. Available: <https://dl.acm.org/doi/10.5555/3454287.3454297>
- [13] D. Narayanan *et al.*, “Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM,” Aug. 2021, doi: 10.48550/arXiv.2104.04473.
- [14] A. Grattafiori *et al.*, “The Llama 3 Herd of Models,” Nov. 2024, doi: 10.48550/arXiv.2407.21783.
- [15] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefler, “Demystifying NCCL: An In-Depth Analysis of GPU Communication Protocols and Algorithms,” in *2025 IEEE Symposium on High-Performance Interconnects (HOTI)*, Aug. 2025, pp. 48–59, doi: 10.1109/HOTI66940.2025.00024.
- [16] NVIDIA, “NVIDIA GB300 NVL72,” Oct. 2025. [Online]. Available: <https://www.nvidia.com/en-us/data-center/gb300-nvl72/>
- [17] —, “Quantum-X800 InfiniBand Switches,” 2025. [Online]. Available: <https://nvdam.widen.net/s/nfdzskhmcn/infiniband-datasheet-quantum-family-3231555>
- [18] Y. Wang, S. Wang, R. Parsons, S. Sanyal, V. Gopal, A. Novick, A. Rizzo, M. Lipson, A. L. Gaeta, and K. Bergman, “Co-Designed Silicon Photonics Chip I/O for Energy-Efficient Petascale Connectivity,” *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 15, no. 8, pp. 1581–1591, Aug. 2025, doi: 10.1109/TCPMT.2024.3492189.
- [19] C. Li, T. Shu, H. Yu, F. Huang, Y. Zhang, F. Qiu, and D. Dai, “Energy-Efficient DWDM Transmitter for Optical I/O enabled by FP-cavity-based PZT optical memristors,” in *2025 Optical Fiber Communications Conference and Exhibition (OFC)*, Mar. 2025, pp. 1–3. [Online]. Available: <https://ieeexplore.ieee.org/document/11046519/>
- [20] NVIDIA, “NVIDIA H100 Tensor Core GPU Architecture,” 2023. [Online]. Available: <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>
- [21] M. Isaev, N. McDonald, L. Dennison, and R. Vuduc, “Calculon: A methodology and tool for high-level co-design of systems and large language models,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. Denver CO USA: ACM, Nov. 2023, pp. 1–14, doi: 10.1145/3581784.3607102.
- [22] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, doi: 10.1145/1498765.1498785.
- [23] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” Jul. 2020, doi: 10.48550/arXiv.2005.14165.
- [24] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 4895–4901, doi: 10.18653/v1/2023.emnlp-main.298.